

Use streaming in ASP.NET Core SignalR

By [Brennan Conroy](#)

ASP.NET Core SignalR supports streaming from client to server and from server to client. This is useful for scenarios where fragments of data arrive over time. When streaming, each fragment is sent to the client or server as soon as it becomes available, rather than waiting for all of the data to become available.

[View or download sample code](#) (how to download)

Set up a hub for streaming

A hub method automatically becomes a streaming hub method when it returns [IAsyncEnumerable<T>](#), [ChannelReader<T>](#), [Task<IAsyncEnumerable<T>>](#), or [Task<ChannelReader<T>>](#).

Server-to-client streaming

Streaming hub methods can return [IAsyncEnumerable<T>](#) in addition to [ChannelReader<T>](#). The simplest way to return [IAsyncEnumerable<T>](#) is by making the hub method an async iterator method as the following sample demonstrates. Hub async iterator methods can accept a [CancellationToken](#) parameter that's triggered when the client unsubscribes from the stream. Async iterator methods avoid problems common with Channels, such as not returning the [ChannelReader](#) early enough or exiting the method without completing the [ChannelWriter<T>](#).

ⓘ Note

The following sample requires C# 8.0 or later.

C#

```
public class AsyncEnumerableHub : Hub
{
    public async IAsyncEnumerable<int> Counter(
        int count,
        int delay,
        [EnumeratorCancellation]
        CancellationToken cancellationToken)
    {
```

```

for (var i = 0; i < count; i++)
{
    // Check the cancellation token regularly so that the server will stop
    // producing items if the client disconnects.
    cancellationToken.ThrowIfCancellationRequested();

    yield return i;

    // Use the cancellationToken in other APIs that accept cancellation
    // tokens so the cancellation can flow down to them.
    await Task.Delay(delay, cancellationToken);
}
}

```

The following sample shows the basics of streaming data to the client using Channels. Whenever an object is written to the `ChannelWriter<T>`, the object is immediately sent to the client. At the end, the `ChannelWriter` is completed to tell the client the stream is closed.

ⓘ Note

Write to the `ChannelWriter<T>` on a background thread and return the `ChannelReader` as soon as possible. Other hub invocations are blocked until a `ChannelReader` is returned.

Wrap logic in a [try... catch statement](#). Complete the `Channel` in a [finally block](#). If you want to flow an error, capture it inside the `catch` block and write it in the `finally` block.

C#

```

public ChannelReader<int> Counter(
    int count,
    int delay,
    CancellationToken cancellationToken)
{
    var channel = Channel.CreateUnbounded<int>();

    // We don't want to await WriteItemsAsync, otherwise we'd end up waiting
    // for all the items to be written before returning the channel back to
    // the client.
    _ = WriteItemsAsync(channel.Writer, count, delay, cancellationToken);

    return channel.Reader;
}

private async Task WriteItemsAsync(
    ChannelWriter<int> writer,
    int count,
    int delay,

```

```
CancellationToken cancellationToken)
{
    Exception localException = null;
    try
    {
        for (var i = 0; i < count; i++)
        {
            await writer.WriteAsync(i, cancellationToken);

            // Use the cancellationToken in other APIs that accept cancellation
            // tokens so the cancellation can flow down to them.
            await Task.Delay(delay, cancellationToken);
        }
    }
    catch (Exception ex)
    {
        localException = ex;
    }
    finally
    {
        writer.Complete(localException);
    }
}
```

Server-to-client streaming hub methods can accept a `CancellationToken` parameter that's triggered when the client unsubscribes from the stream. Use this token to stop the server operation and release any resources if the client disconnects before the end of the stream.

Client-to-server streaming

A hub method automatically becomes a client-to-server streaming hub method when it accepts one or more objects of type `ChannelReader<T>` or `IAsyncEnumerable<T>`. The following sample shows the basics of reading streaming data sent from the client. Whenever the client writes to the `ChannelWriter<T>`, the data is written into the `ChannelReader` on the server from which the hub method is reading.

C#

```
public async Task UploadStream(ChannelReader<string> stream)
{
    while (await stream.WaitToReadAsync())
    {
        while (stream.TryRead(out var item))
        {
            // do something with the stream item
            Console.WriteLine(item);
        }
    }
}
```

```
}  
}
```

An `IAsyncEnumerable<T>` version of the method follows.

ⓘ Note

The following sample requires C# 8.0 or later.

C#

```
public async Task UploadStream(IAsyncEnumerable<string> stream)  
{  
    await foreach (var item in stream)  
    {  
        Console.WriteLine(item);  
    }  
}
```

.NET client

Server-to-client streaming

The `StreamAsync` and `StreamAsChannelAsync` methods on `HubConnection` are used to invoke server-to-client streaming methods. Pass the hub method name and arguments defined in the hub method to `StreamAsync` or `StreamAsChannelAsync`. The generic parameter on `StreamAsync<T>` and `StreamAsChannelAsync<T>` specifies the type of objects returned by the streaming method. An object of type `IAsyncEnumerable<T>` or `ChannelReader<T>` is returned from the stream invocation and represents the stream on the client.

A `StreamAsync` example that returns `IAsyncEnumerable<int>`:

C#

```
// Call "Cancel" on this CancellationTokensource to send a cancellation message to  
// the server, which will trigger the corresponding token in the hub method.  
var cancellationTokensource = new CancellationTokensource();  
var stream = hubConnection.StreamAsync<int>(  
    "Counter", 10, 500, cancellationTokensource.Token);  
  
await foreach (var count in stream)  
{  
    Console.WriteLine($"{count}");  
}
```

```
Console.WriteLine("Streaming completed");
```

A corresponding `StreamAsChannelAsync` example that returns `ChannelReader<int>`:

C#

```
// Call "Cancel" on this CancellationTokensource to send a cancellation message to
// the server, which will trigger the corresponding token in the hub method.
var cancellationTokensource = new CancellationTokensource();
var channel = await hubConnection.StreamAsChannelAsync<int>(
    "Counter", 10, 500, cancellationTokensource.Token);

// Wait asynchronously for data to become available
while (await channel.WaitToReadAsync())
{
    // Read all currently available data synchronously, before waiting for more data
    while (channel.TryRead(out var count))
    {
        Console.WriteLine($"{count}");
    }
}

Console.WriteLine("Streaming completed");
```

In the previous code:

- The `StreamAsChannelAsync` method on `HubConnection` is used to invoke a server-to-client streaming method. Pass the hub method name and arguments defined in the hub method to `StreamAsChannelAsync`.
- The generic parameter on `StreamAsChannelAsync<T>` specifies the type of objects returned by the streaming method.
- A `ChannelReader<T>` is returned from the stream invocation and represents the stream on the client.

Client-to-server streaming

There are two ways to invoke a client-to-server streaming hub method from the .NET client. You can either pass in an `IAsyncEnumerable<T>` or a `ChannelWriter` as an argument to `SendAsync`, `InvokeAsync`, or `StreamAsChannelAsync`, depending on the hub method invoked.

Whenever data is written to the `IAsyncEnumerable` or `ChannelWriter` object, the hub method on the server receives a new item with the data from the client.

If using an `IAsyncEnumerable` object, the stream ends after the method returning stream items exits.

ⓘ Note

The following sample requires C# 8.0 or later.

C#

```
async IAsyncEnumerable<string> clientStreamData()
{
    for (var i = 0; i < 5; i++)
    {
        var data = await FetchSomeData();
        yield return data;
    }
    //After the for loop has completed and the local function exits the stream completion will be sent.
}

await connection.SendAsync("UploadStream", clientStreamData());
```

Or if you're using a `ChannelWriter`, you complete the channel with `channel.Writer.Complete()`:

C#

```
var channel = Channel.CreateBounded<string>(10);
await connection.SendAsync("UploadStream", channel.Reader);
await channel.Writer.WriteAsync("some data");
await channel.Writer.WriteAsync("some more data");
channel.Writer.Complete();
```

JavaScript client

Server-to-client streaming

JavaScript clients call server-to-client streaming methods on hubs with `connection.stream`. The `stream` method accepts two arguments:

- The name of the hub method. In the following example, the hub method name is `Counter`.
- Arguments defined in the hub method. In the following example, the arguments are a count for the number of stream items to receive and the delay between stream items.

`connection.stream` returns an `IStreamResult`, which contains a `subscribe` method. Pass an `IStreamSubscriber` to `subscribe` and set the `next`, `error`, and `complete` callbacks to receive notifications from the `stream` invocation.

JavaScript

```
connection.stream("Counter", 10, 500)
    .subscribe({
        next: (item) => {
            var li = document.createElement("li");
            li.textContent = item;
            document.getElementById("messagesList").appendChild(li);
        },
        complete: () => {
            var li = document.createElement("li");
            li.textContent = "Stream completed";
            document.getElementById("messagesList").appendChild(li);
        },
        error: (err) => {
            var li = document.createElement("li");
            li.textContent = err;
            document.getElementById("messagesList").appendChild(li);
        },
    });
```

To end the stream from the client, call the `dispose` method on the `ISubscription` that's returned from the `subscribe` method. Calling this method causes cancellation of the `CancellationToken` parameter of the Hub method, if you provided one.

Client-to-server streaming

JavaScript clients call client-to-server streaming methods on hubs by passing in a `Subject` as an argument to `send`, `invoke`, or `stream`, depending on the hub method invoked. The `Subject` is a class that looks like a `Subject`. For example in RxJS, you can use the `Subject` class from that library.

JavaScript

```
const subject = new signalR.Subject();
yield connection.send("UploadStream", subject);
var iteration = 0;
const intervalHandle = setInterval(() => {
    iteration++;
    subject.next(iteration.toString());
    if (iteration === 10) {
        clearInterval(intervalHandle);
        subject.complete();
    }
}, 1000);
```

```
    }  
    }, 500);
```

Calling `subject.next(item)` with an item writes the item to the stream, and the hub method receives the item on the server.

To end the stream, call `subject.complete()`.

Java client

Server-to-client streaming

The SignalR Java client uses the `stream` method to invoke streaming methods. `stream` accepts three or more arguments:

- The expected type of the stream items.
- The name of the hub method.
- Arguments defined in the hub method.

Java

```
hubConnection.stream(String.class, "ExampleStreamingHubMethod", "Arg1")  
    .subscribe(  
        (item) -> { /* Define your onNext handler here. */ },  
        (error) -> { /* Define your onError handler here. */ },  
        () -> { /* Define your onCompleted handler here. */ });
```

The `stream` method on `HubConnection` returns an `Observable` of the stream item type. The `Observable` type's `subscribe` method is where `onNext`, `onError` and `onCompleted` handlers are defined.

Client-to-server streaming

The SignalR Java client can call client-to-server streaming methods on hubs by passing in an [Observable](#) as an argument to `send`, `invoke`, or `stream`, depending on the hub method invoked.

Java

```
ReplaySubject<String> stream = ReplaySubject.create();  
hubConnection.send("UploadStream", stream);  
stream.onNext("FirstItem");
```



```
stream.onNext("SecondItem");  
stream.onComplete();
```

Calling `stream.onNext(item)` with an item writes the item to the stream, and the hub method receives the item on the server.

To end the stream, call `stream.onComplete()`.

Additional resources

- [Hubs](#)
- [.NET client](#)
- [JavaScript client](#)
- [Publish to Azure](#)

Last updated on 08/28/2025